

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
ĐẠI HỌC QUỐC GIA HÀ NỘI

BÁO CÁO BÀI TẬP LỚN
TRÒ CHƠI CỜ CARO VỚI AI
Thuật toán Minimax và Alpha-Beta Pruning

Môn học: Cơ sở Trí tuệ Nhân tạo

Mục lục

1	Mô tả bài toán cờ Caro	3
1.1	Đặc điểm bài toán	3
1.2	Mục tiêu	3
2	Luật chơi và điều kiện thắng	3
2.1	Luật chơi	3
2.2	Điều kiện thắng	4
2.3	Điều kiện hòa	4
3	Cách biểu diễn trạng thái bàn cờ	4
3.1	Cấu trúc dữ liệu	4
3.2	Cài đặt trong mã nguồn	4
3.3	Ưu điểm của cách biểu diễn	5
4	Cách sinh nước đi hợp lệ	5
4.1	Phương pháp 1: Sinh tất cả nước đi (<code>get_all_valid_moves</code>)	5
4.2	Phương pháp 2: Sinh nước đi lân cận (<code>get_valid_moves</code>)	5
4.3	Phân tích hiệu quả	6
5	Cách kiểm tra trạng thái kết thúc	6
5.1	Kiểm tra chiến thắng (<code>check_win</code>)	6
5.2	Kiểm tra hòa	7
5.3	Độ phức tạp	7
6	Thuật toán Minimax đã cài đặt	7
6.1	Ý tưởng thuật toán	7
6.2	Mã giả	8
6.3	Cài đặt trong mã nguồn	8
6.4	Độ phức tạp	9
7	Thuật toán Alpha-Beta Pruning đã cài đặt	9
7.1	Ý tưởng cải tiến	9
7.2	Mã giả	10
7.3	Cài đặt trong mã nguồn	10
7.4	Độ phức tạp	11
8	Hàm đánh giá trạng thái	11
8.1	Hàm đánh giá chính (<code>evaluate_simple</code>)	12

8.2	Hàm đánh giá nâng cao (<code>evaluate_board</code>)	13
8.3	Nguyên tắc thiết kế hàm đánh giá	13
9	Thiết kế các trạng thái thử nghiệm	14
9.1	State 1: Bàn cờ trống	14
9.2	State 2: Giữa trận (5 nước đi)	14
9.3	State 3: O có 3 quân liên tiếp (hàng ngang)	14
9.4	State 4: X có 3 quân liên tiếp (hàng ngang)	15
9.5	State 5: Quân rải rác	15
9.6	State 6: Phức tạp (9 quân)	15
10	Bảng kết quả thực nghiệm	15
11	Nhận xét về số trạng thái đã xét và thời gian chạy	17
11.1	Số trạng thái đã xét (Nodes visited)	17
11.2	Thời gian chạy	18
12	Nhận xét về ảnh hưởng của độ sâu tìm kiếm	18
12.1	Lựa chọn thuật toán theo mức độ khó	18
12.2	Chất lượng nước đi	19
12.3	Mối quan hệ giữa độ sâu và tài nguyên	19
12.4	Kết luận	20
13	Ưu điểm và hạn chế của chương trình	20
13.1	Ưu điểm	20
13.2	Hạn chế	20
13.3	Hướng phát triển	21
14	Tài liệu tham khảo và phần đã tham khảo từ các repo mẫu	21
14.1	Tài liệu tham khảo	21
14.2	Phần đã tham khảo từ repo mẫu	21

1 Mô tả bài toán cờ Caro

Cờ Caro (hay còn gọi là Gomoku, Five-in-a-Row) là một trò chơi chiến thuật hai người trên bàn cờ kẻ ô vuông. Đây là một bài toán kinh điển trong lĩnh vực Trí tuệ Nhân tạo, thuộc nhóm **trò chơi đối kháng hai người với thông tin đầy đủ** (two-player zero-sum game with perfect information).

1.1 Đặc điểm bài toán

- **Loại bài toán:** Tìm kiếm đối kháng (Adversarial Search).
- **Không gian trạng thái:** Cực lớn – với bàn cờ 9×9 , số trạng thái lý thuyết lên tới $3^{81} \approx 4.4 \times 10^{38}$.
- **Thông tin đầy đủ:** Cả hai người chơi đều nhìn thấy toàn bộ bàn cờ tại mọi thời điểm.
- **Tổng bằng không (Zero-sum):** Một bên thắng thì bên kia thua, lợi ích của hai bên đối lập nhau.
- **Xác định (Deterministic):** Không có yếu tố ngẫu nhiên trong trò chơi.

1.2 Mục tiêu

Xây dựng chương trình chơi Cờ Caro trên bàn cờ kích thước 9×9 , trong đó:

1. Người chơi đối đầu với AI (máy tính).
2. AI sử dụng thuật toán Minimax và Alpha-Beta Pruning để tìm nước đi tối ưu.
3. Có giao diện đồ họa thân thiện sử dụng Tkinter.
4. Hỗ trợ nhiều mức độ khó (điều chỉnh độ sâu tìm kiếm).

2 Luật chơi và điều kiện thắng

2.1 Luật chơi

1. Bàn cờ có kích thước 9×9 ô vuông.
2. Hai người chơi lần lượt đặt quân:
 - Người chơi (Human) sử dụng quân **X** và đi trước.
 - Máy tính (AI) sử dụng quân **O** và đi sau.
3. Mỗi lượt, người chơi đặt quân của mình vào một ô trống bất kỳ.
4. Quân đã đặt không được di chuyển hay thu hồi.

2.2 Điều kiện thắng

Người chơi chiến thắng khi tạo được chuỗi **4 quân liên tiếp** theo một trong bốn hướng:

- Hàng ngang (horizontal): $\rightarrow$
- Hàng dọc (vertical): $\downarrow$
- Đường chéo chính (diagonal): $\searrow$
- Đường chéo phụ (anti-diagonal): $\swarrow$

2.3 Điều kiện hòa

Trận đấu hòa khi toàn bộ $9 \times 9 = 81$ ô trên bàn cờ đã được lấp đầy mà không có bên nào đạt được 4 quân liên tiếp.

Lưu ý: Trong phiên bản này, điều kiện thắng là **4 quân liên tiếp** (thay vì 5 như Gomoku chuẩn) để phù hợp với kích thước bàn cờ 9×9 thu nhỏ.

3 Cách biểu diễn trạng thái bàn cờ

3.1 Cấu trúc dữ liệu

Bàn cờ được biểu diễn bằng một **ma trận 2 chiều** (mảng 2D) kích thước 9×9 , trong đó mỗi ô nhận một trong ba giá trị:

Bảng 1: Giá trị biểu diễn trạng thái ô cờ

Giá trị	Hằng số	Ý nghĩa
0	EMPTY	Ô trống, chưa có quân
1	PLAYER_X	Quân X (người chơi)
-1	PLAYER_O	Quân O (AI)

3.2 Cài đặt trong mã nguồn

```
1 BOARD_SIZE = 9
2 EMPTY = 0
3 PLAYER_X = 1      # Người chơi
4 PLAYER_O = -1     # AI
5
6 class CaroGame:
7     def __init__(self, size=BOARD_SIZE):
8         self.size = size
9         self.board = [[EMPTY for _ in range(size)] for _ in
10                        range(size)]
11         self.current_player = PLAYER_X  # X đi trước
```

```

11         self.last_move = None           # Lưu nước đi cuối cùng
12         self.move_count = 0             # Đếm số nước đã đi
13         self.winner = None              # Người thắng
14         self.game_over = False          # Trạng thái kết thúc

```

Listing 1: Biểu diễn trạng thái bàn cờ (caro_game.py)

3.3 Ưu điểm của cách biểu diễn

- Sử dụng giá trị 1 và -1 cho hai người chơi giúp dễ dàng đổi lượt bằng phép nhân -1: `current_player = -current_player`.
- Xác định đối thủ đơn giản: `opponent = -player`.
- Truy cập nhanh $O(1)$ vào bất kỳ ô nào trên bàn cờ.
- Phương thức `copy()` sử dụng `deepcopy` để tạo bản sao trạng thái cho cây tìm kiếm.

4 Cách sinh nước đi hợp lệ

Việc sinh nước đi hợp lệ là yếu tố then chốt ảnh hưởng đến hiệu suất của thuật toán tìm kiếm. Chương trình cài đặt hai phương pháp:

4.1 Phương pháp 1: Sinh tất cả nước đi (get_all_valid_moves)

Liệt kê toàn bộ ô trống trên bàn cờ. Đây là cách đơn giản nhưng **không hiệu quả** vì phải xét tối đa 81 ô:

```

1 def get_all_valid_moves(self):
2     moves = []
3     for i in range(self.size):
4         for j in range(self.size):
5             if self.board[i][j] == EMPTY:
6                 moves.append((i, j))
7     return moves

```

Listing 2: Sinh tất cả nước đi hợp lệ

4.2 Phương pháp 2: Sinh nước đi lân cận (get_valid_moves)

Chỉ xét các ô trống **liền kề** (8 hướng) với các quân đã đặt trên bàn cờ. Cách tiếp cận này giảm đáng kể số nước đi cần xét.

```

1 def get_valid_moves(self):
2     if self.move_count == 0:
3         return [(self.size // 2, self.size // 2)] # Nước đầu o
4         trung tam

```

```

5 moves = set()
6 for i in range(self.size):
7     for j in range(self.size):
8         if self.board[i][j] != EMPTY:
9             for di in [-1, 0, 1]:
10                 for dj in [-1, 0, 1]:
11                     if di == 0 and dj == 0:
12                         continue
13                     ni, nj = i + di, j + dj
14                     if self.is_valid_move(ni, nj):
15                         moves.add((ni, nj))
16
17 # Sap xep theo khoang cach toi nuoc di cuoi cung
18 if self.last_move:
19     last_r, last_c = self.last_move
20     moves = sorted(moves,
21                     key=lambda m: abs(m[0]-last_r) + abs(m[1]-last_c))
22 return list(moves)

```

Listing 3: Sinh nước đi lên cân – chiến lược tối ưu

4.3 Phân tích hiệu quả

- **Nước đi đầu tiên:** Luôn trả về ô trung tâm (4, 4) – đây là vị trí chiến lược nhất.
- **Giới hạn không gian tìm kiếm:** Thay vì xét tất cả ô trống (tối đa 81), chỉ xét các ô lân cận quân đã đặt (thường 10–25 ô).
- **Sắp xếp theo khoảng cách Manhattan:** Ưu tiên các nước đi gần nước đi cuối cùng, giúp cải thiện hiệu quả cắt tỉa Alpha-Beta.

5 Cách kiểm tra trạng thái kết thúc

Trạng thái kết thúc được kiểm tra sau mỗi nước đi. Chương trình kiểm tra hai điều kiện:

5.1 Kiểm tra chiến thắng (check_win)

Sau mỗi nước đi tại vị trí (row, col) , kiểm tra 4 hướng (ngang, dọc, chéo chính, chéo phụ). Đếm số quân liên tiếp cùng loại theo cả hai chiều dọc theo mỗi trục:

```
1 def check_win(self, row, col, player):  
2     directions = [(0, 1), (1, 0), (1, 1), (1, -1)]  
3     for d_row, d_col in directions:  
4         count = 1  
5         count += self.count_direction(row, col, d_row, d_col,  
6                                     player)  
7         count += self.count_direction(row, col, -d_row, -d_col,  
8                                     player)  
9         if count >= 4:
```

```

8         return True
9     return False

```

Listing 4: Kiểm tra chiến thắng

```

1 def count_direction(self, row, col, d_row, d_col, player):
2     count = 0
3     r, c = row + d_row, col + d_col
4     while (0 <= r < self.size and 0 <= c < self.size
5           and self.board[r][c] == player):
6         count += 1
7         r += d_row
8         c += d_col
9     return count

```

Listing 5: Đếm quân liên tiếp theo một hướng

5.2 Kiểm tra hòa

Trận đấu hòa khi tổng số nước đi bằng $9 \times 9 = 81$ mà không ai thắng:

```

1 def make_move(self, row, col, player=None):
2     # ... dat quan ...
3     if self.check_win(row, col, player):
4         self.winner = player
5         self.game_over = True
6     elif self.move_count >= self.size * self.size:
7         self.winner = 0          # Hoa
8         self.game_over = True

```

Listing 6: Kiểm tra trạng thái kết thúc tổng quát

5.3 Độ phức tạp

Kiểm tra chiến thắng có độ phức tạp $O(k)$ với k là độ dài chuỗi thắng (ở đây $k = 4$). Nhờ chỉ kiểm tra tại vị trí vừa đặt quân, thuật toán rất hiệu quả so với việc quét toàn bộ bàn cờ.

6 Thuật toán Minimax đã cài đặt

6.1 Ý tưởng thuật toán

Minimax là thuật toán tìm kiếm trên cây trò chơi dựa trên nguyên tắc:

- Người chơi **MAX** (AI - quân O) luôn chọn nước đi có giá trị **cao nhất**.
- Người chơi **MIN** (Human - quân X) luôn chọn nước đi có giá trị **thấp nhất**.
- Cả hai bên đều chơi **tối ưu** (optimal play).

6.2 Mã giả

Thuật toán 1 Thuật toán Minimax

Đầu vào: Trạng thái game, độ sâu d , lượt chơi (MAX/MIN)

Đầu ra: Giá trị đánh giá tốt nhất

```

1: function MINIMAX(game, depth, isMaximizing)
2:   if game là trạng thái kết thúc then
3:     return giá trị kết quả ( $\pm 100000$  hoặc 0)
4:   end if
5:   if depth = 0 then
6:     return evaluate(game)                                ▷ Đánh giá heuristic
7:   end if
8:   if isMaximizing then
9:     maxVal  $\leftarrow -\infty$ 
10:    for all nước đi hợp lệ move do
11:      Thực hiện move trên bản sao game
12:      val  $\leftarrow$  MINIMAX(game', depth - 1, False)
13:      maxVal  $\leftarrow$  max(maxVal, val)
14:    end for
15:    return maxVal
16:  else
17:    minVal  $\leftarrow +\infty$ 
18:    for all nước đi hợp lệ move do
19:      Thực hiện move trên bản sao game
20:      val  $\leftarrow$  MINIMAX(game', depth - 1, True)
21:      minVal  $\leftarrow$  min(minVal, val)
22:    end for
23:    return minVal
24:  end if
25: end function

```

6.3 Cài đặt trong mã nguồn

```

1 def minimax(self, game, depth, is_maximizing):
2     self.nodes_visited += 1
3
4     if game.is_terminal():
5         result = game.get_result()
6         if result == PLAYER_0:      # AI thắng
7             return 100000
8         elif result == PLAYER_X:    # Người chơi thắng
9             return -100000
10        else:
11            return 0                  # Hòa
12
13    if depth == 0:
14        return self.evaluation_func(game, PLAYER_0)

```

```

15
16     valid_moves = game.get_valid_moves()
17     if not valid_moves:
18         return 0
19
20     if is_maximizing:
21         max_val = -math.inf
22         for move in valid_moves:
23             row, col = move
24             game_copy = game.copy()
25             game_copy.make_move(row, col)
26             val = self.minimax(game_copy, depth - 1, False)
27             max_val = max(max_val, val)
28         return max_val
29     else:
30         min_val = math.inf
31         for move in valid_moves:
32             row, col = move
33             game_copy = game.copy()
34             game_copy.make_move(row, col)
35             val = self.minimax(game_copy, depth - 1, True)
36             min_val = min(min_val, val)
37         return min_val

```

Listing 7: Cài đặt thuật toán Minimax (ai_agent.py)

6.4 Độ phức tạp

- **Thời gian:** $O(b^d)$ với b là hệ số phân nhánh (branching factor, trung bình 10–25 nước đi), d là độ sâu tìm kiếm.
- **Không gian:** $O(b \cdot d)$ cho ngăn xếp đệ quy.

7 Thuật toán Alpha-Beta Pruning đã cài đặt

7.1 Ý tưởng cải tiến

Alpha-Beta Pruning là cải tiến của Minimax, sử dụng hai tham số α và β để **cắt tỉa** các nhánh cây không cần thiết:

- α : Giá trị tốt nhất mà MAX đã tìm được (lower bound).
- β : Giá trị tốt nhất mà MIN đã tìm được (upper bound).
- **Điều kiện cắt tỉa:** Khi $\beta \leq \alpha$, dừng duyệt nhánh hiện tại vì nhánh này không thể cải thiện kết quả.

7.2 Mã giả

Thuật toán 2 Thuật toán Alpha-Beta Pruning

Đầu vào: Trạng thái game, độ sâu d , α , β , lượt chơi

Đầu ra: Giá trị đánh giá tốt nhất

```

1: function ALPHABETA( $game, depth, \alpha, \beta, isMaximizing$ )
2:   if  $game$  là trạng thái kết thúc or  $depth = 0$  then
3:     return giá trị đánh giá
4:   end if
5:   if  $isMaximizing$  then
6:      $maxVal \leftarrow -\infty$ 
7:     for all nước đi hợp lệ  $move$  do
8:        $val \leftarrow$  ALPHABETA( $game', depth - 1, \alpha, \beta, False$ )
9:        $maxVal \leftarrow \max(maxVal, val)$ 
10:       $\alpha \leftarrow \max(\alpha, val)$ 
11:      if  $\beta \leq \alpha$  then                                ▷ Cắt tỉa Beta
12:        break
13:      end if
14:    end for
15:    return  $maxVal$ 
16:  else
17:     $minVal \leftarrow +\infty$ 
18:    for all nước đi hợp lệ  $move$  do
19:       $val \leftarrow$  ALPHABETA( $game', depth - 1, \alpha, \beta, True$ )
20:       $minVal \leftarrow \min(minVal, val)$ 
21:       $\beta \leftarrow \min(\beta, val)$ 
22:      if  $\beta \leq \alpha$  then                                ▷ Cắt tỉa Alpha
23:        break
24:      end if
25:    end for
26:    return  $minVal$ 
27:  end if
28: end function

```

7.3 Cài đặt trong mã nguồn

```

1 def alpha_beta(self, game, depth, alpha, beta, is_maximizing):
2     self.nodes_visited += 1
3
4     if game.is_terminal():
5         result = game.get_result()
6         if result == PLAYER_0:
7             return 100000
8         elif result == PLAYER_X:
9             return -100000
10        else:
11            return 0

```

```

12
13     if depth == 0:
14         return self.evaluation_func(game, PLAYER_0)
15
16     valid_moves = game.get_valid_moves()
17     if not valid_moves:
18         return 0
19
20     if is_maximizing:
21         max_val = -math.inf
22         for move in valid_moves:
23             row, col = move
24             game_copy = game.copy()
25             game_copy.make_move(row, col)
26             val = self.alpha_beta(game_copy, depth-1, alpha,
27                                   beta, False)
28             max_val = max(max_val, val)
29             alpha = max(alpha, val)
30             if beta <= alpha:
31                 break # Cat tia Beta
32         return max_val
33     else:
34         min_val = math.inf
35         for move in valid_moves:
36             row, col = move
37             game_copy = game.copy()
38             game_copy.make_move(row, col)
39             val = self.alpha_beta(game_copy, depth-1, alpha,
40                                   beta, True)
41             min_val = min(min_val, val)
42             beta = min(beta, val)
43             if beta <= alpha:
44                 break # Cat tia Alpha
45         return min_val

```

Listing 8: Cài đặt Alpha-Beta Pruning (ai_agent.py)

7.4 Độ phức tạp

- Trường hợp tốt nhất: $O(b^{d/2})$ – giảm một nửa độ sâu hiệu quả so với Minimax.
- Trường hợp xấu nhất: $O(b^d)$ – tương đương Minimax khi không cắt tỉa được.
- Trung bình: $O(b^{3d/4})$ – cải thiện đáng kể so với Minimax thuần.

8 Hàm đánh giá trạng thái

Hàm đánh giá (evaluation function) là thành phần quan trọng nhất, quyết định “trí thông minh” của AI. Chương trình cài đặt hai phiên bản hàm đánh giá.

8.1 Hàm đánh giá chính (evaluate_simple)

Hàm này duyệt toàn bộ bàn cờ, tại mỗi ô có quân, kiểm tra 4 hướng và đánh giá dựa trên:

- Số quân liên tiếp cùng loại.
- Số đầu bị chặn bởi quân đối phương.

Bảng 2: Bảng điểm theo mẫu hình (Pattern Scores)

Mẫu hình	Số quân	Đầu bị chặn	Điểm
Chiến thắng	≥ 4	bất kỳ	100.000
Mở 3 quân (live three)	3	0	1.000
Chặn 1 đầu 3 quân	3	1	100
Mở 2 quân (live two)	2	0	100
Chặn 1 đầu 2 quân	2	1	10
Quân đơn lẻ	1	0	1

```

1 def evaluate_simple(game, player):
2     if game.is_terminal():
3         result = game.get_result()
4         if result == player:
5             return 100000
6         elif result == -player:
7             return -100000
8         else:
9             return 0
10
11     score = 0
12     directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
13
14     for i in range(BOARD_SIZE):
15         for j in range(BOARD_SIZE):
16             if game.board[i][j] == EMPTY:
17                 continue
18             cell_player = game.board[i][j]
19             for d_row, d_col in directions:
20                 count = 1
21                 blocked = 0
22                 # Dem theo huong thuan
23                 r, c = i + d_row, j + d_col
24                 while 0 <= r < BOARD_SIZE and 0 <= c <
25                     BOARD_SIZE:
26                     if game.board[r][c] == cell_player:
27                         count += 1; r += d_row; c += d_col
28                     elif game.board[r][c] == -cell_player:
29                         blocked += 1; break
30                     else:
31                         break

```

```

31         # Dem theo huong nguoc
32         r, c = i - d_row, j - d_col
33         while 0 <= r < BOARD_SIZE and 0 <= c <
           BOARD_SIZE:
34             if game.board[r][c] == cell_player:
35                 count += 1; r -= d_row; c -= d_col
36             elif game.board[r][c] == -cell_player:
37                 blocked += 1; break
38             else:
39                 break
40         # Tinh diem theo mau hinh
41         if count >= 4:             pattern_score = 100000
42         elif count==3 and blocked==0: pattern_score =
           1000
43         elif count==3 and blocked==1: pattern_score = 100
44         elif count==2 and blocked==0: pattern_score = 100
45         elif count==2 and blocked==1: pattern_score = 10
46         elif count==1 and blocked==0: pattern_score = 1
47         else: pattern_score = 0
48         # Cong diem cho AI, tru diem cho doi thu
49         if cell_player == player:
50             score += pattern_score
51         else:
52             score -= pattern_score
53     return score

```

Listing 9: Hàm đánh giá chính (evaluation.py)

8.2 Hàm đánh giá nâng cao (evaluate_board)

Hàm này bổ sung thêm **bonus vị trí trung tâm** – các quân ở gần trung tâm bàn cờ được cộng thêm điểm:

$$\text{center_bonus} = \max(0, 10 - d_{\text{Manhattan}}) \times 5$$

trong đó $d_{\text{Manhattan}} = |i - 4| + |j - 4|$ là khoảng cách Manhattan đến ô trung tâm.

8.3 Nguyên tắc thiết kế hàm đánh giá

1. **Đối xứng:** Điểm AI cộng, điểm đối thủ trừ $\Rightarrow$ tổng bằng 0 khi cân bằng.
2. **Ưu tiên tấn công:** Các mẫu hình tấn công (mở 3, mở 2) được đánh giá cao.
3. **Phòng thủ tự động:** Vì hàm đánh giá cũng trừ điểm cho mẫu hình đối thủ, AI sẽ tự động phòng thủ khi đối thủ có thế mạnh.
4. **Bonus vị trí:** Khuyến khích AI chiếm giữ vùng trung tâm bàn cờ.

9 Thiết kế các trạng thái thử nghiệm

Chương trình xây dựng 6 trạng thái thử nghiệm với mức độ phức tạp tăng dần để đánh giá hiệu suất AI.

9.1 State 1: Bàn cờ trống

Bàn cờ hoàn toàn trống, AI đi nước đầu tiên. Dùng để kiểm tra chiến lược mở đầu.

[illegible]

9.2 State 2: Giữa trận (5 nước đi)

Trạng thái giữa trận với cả hai bên đã đặt quân. X có 3 quân, O có 2 quân.

[illegible]

9.3 State 3: O có 3 quân liên tiếp (hàng ngang)

AI (O) có 3 quân liên tiếp, kiểm tra khả năng tấn công hoàn thành chuỗi 4.

```

    0 1 2 3 4 5 6 7 8
0 . . . . . . . .
...
4 . . . . 0 0 0 . .   0: (4,4), (4,5), (4,6)
...                   Luot cua 0

```

9.4 State 4: X có 3 quân liên tiếp (hàng ngang)

Người chơi (X) có 3 quân liên tiếp, kiểm tra khả năng phòng thủ của AI.

	0	1	2	3	4	5	6	7	8	
0	.	.	.	.	.	.	.	.	.	
	...									
3	.	.	.	X	X	X	.	.	.	X: (3,3), (3,4), (3,5)
	...									

Luot của 0 (phai chan)

9.5 State 5: Quân rải rác

Cả hai bên có quân rải rác ở hai vùng khác nhau trên bàn cờ.

	0	1	2	3	4	5	6	7	8	
0	.	.	.	.	.	.	.	.	.	
	...									
2	.	.	X	X	.	.	.	.	.	X: (2,2), (2,3)
	...									
5	.	.	.	.	.	0	.	.	.	0: (5,5), (6,5)
6	.	.	.	.	.	0	.	.	.	Luot của 0
	...									

9.6 State 6: Phức tạp (9 quân)

Trạng thái phức tạp nhất với 9 quân đã đặt, nhiều mẫu hình đan xen.

	0	1	2	3	4	5	6	7	8	
0	X	.	.	.	.	.	.	0	.	X: (0,0), (1,1), (2,2), (4,0), (5,0)
1	.	X	.	.	.	.	.	0	.	0: (7,7), (6,6), (0,7), (1,7), (2,7)
2	.	.	X	.	.	.	.	0	.	Luot của 0
3	.	.	.	.	.	.	.	.	.	
4	X	.	.	.	.	.	.	.	.	
5	X	.	.	.	.	.	.	.	.	
6	.	.	.	.	.	.	0	.	.	
7	.	.	.	.	.	.	.	0	.	
8	.	.	.	.	.	.	.	.	.	

10 Bảng kết quả thực nghiệm

Kết quả thực nghiệm được thu thập bằng cách chạy cả hai thuật toán Minimax và Alpha-Beta Pruning trên 6 trạng thái thử nghiệm với các độ sâu từ 1 đến 4. Các số liệu bao gồm: số trạng thái đã duyệt (Nodes) và thời gian chạy (Time).

Bảng 3: Kết quả thực nghiệm – State 1 (Bàn cờ trống, lượt X)

Depth	Minimax			Alpha-Beta			Tỷ lệ cắt tỉa
	Best Move	Nodes	Time (s)	Best Move	Nodes	Time (s)	
1	(4,4)	2	<0.001	(4,4)	2	0.002	1.00x
2	(4,4)	10	<0.001	(4,4)	10	<0.001	1.00x
3	(4,4)	98	0.004	(4,4)	52	0.002	1.88x
4	(4,4)	1.298	0.054	(4,4)	341	0.015	3.81x

Bảng 4: Kết quả thực nghiệm – State 2 (Giữa trận, 5 nước đi, lượt O)

Depth	Minimax			Alpha-Beta			Tỷ lệ cắt tỉa
	Best Move	Nodes	Time (s)	Best Move	Nodes	Time (s)	
1	(2,4)	32	<0.001	(2,4)	32	<0.001	1.00x
2	(2,4)	324	0.014	(2,4)	120	0.006	2.70x
3	(5,4)	6.288	0.317	(5,4)	1.176	0.062	5.35x
4	(2,4)	140.324	7.752	(2,4)	9.023	0.502	15.55x

Bảng 5: Kết quả thực nghiệm – State 3 (O có 3 quân hàng ngang, lượt O)

Depth	Minimax			Alpha-Beta			Tỷ lệ cắt tỉa
	Best Move	Nodes	Time (s)	Best Move	Nodes	Time (s)	
1	(4,3)	24	0.002	(4,3)	24	<0.001	1.00x
2	(4,3)	172	0.011	(4,3)	85	0.004	2.02x
3	(3,3)	2.698	0.154	(3,3)	306	0.018	8.82x
4	(3,3)	45.940	2.369	(3,3)	1.315	0.074	34.94x

Bảng 6: Kết quả thực nghiệm – State 4 (X có 3 quân hàng ngang, lượt O phòng thủ)

Depth	Minimax			Alpha-Beta			Tỷ lệ cắt tỉa
	Best Move	Nodes	Time (s)	Best Move	Nodes	Time (s)	
1	(3,2)	24	0.001	(3,2)	24	<0.001	1.00x
2	(2,2)	200	0.008	(2,2)	117	0.004	1.71x
3	(2,2)	2.848	0.139	(2,2)	809	0.046	3.52x
4	(2,2)	54.121	2.763	(2,2)	5.779	0.285	9.37x

Bảng 7: Kết quả thực nghiệm – State 5 (Quân rải rác, lượt O)

Depth	Minimax			Alpha-Beta			Tỷ lệ cắt tỉa
	Best Move	Nodes	Time (s)	Best Move	Nodes	Time (s)	
1	(4,5)	40	0.002	(4,5)	40	<0.001	1.00x
2	(4,5)	490	0.026	(4,5)	246	0.011	1.99x
3	(4,5)	11.561	0.605	(4,5)	1.849	0.095	6.25x
4	(4,5)	302.295	16.074	(4,5)	11.920	0.671	25.36x

Bảng 8: Kết quả thực nghiệm – State 6 (Phức tạp, 9 quân, lượt O)

Depth	Minimax			Alpha-Beta			Tỷ lệ cắt tỉa
	Best Move	Nodes	Time (s)	Best Move	Nodes	Time (s)	
1	(3,7)	74	0.001	(3,7)	74	0.002	1.00x
2	(3,7)	1.437	0.084	(3,7)	359	0.021	4.00x
3	(3,7)	52.839	3.198	(3,7)	4.354	0.252	12.14x
4	(3,7)	2.032.724	130.429	(3,7)	39.831	2.519	51.03x

Bảng 9: Bảng tổng hợp – Tỷ lệ cắt tỉa Alpha-Beta so với Minimax (theo Nodes)

Depth	State 1	State 2	State 3	State 4	State 5	State 6
1	1.00x	1.00x	1.00x	1.00x	1.00x	1.00x
2	1.00x	2.70x	2.02x	1.71x	1.99x	4.00x
3	1.88x	5.35x	8.82x	3.52x	6.25x	12.14x
4	3.81x	15.55x	34.94x	9.37x	25.36x	51.03x

11 Nhận xét về số trạng thái đã xét và thời gian chạy

11.1 Số trạng thái đã xét (Nodes visited)

- Minimax:** Số trạng thái tăng theo hàm mũ $O(b^d)$. Ví dụ ở State 6 (phức tạp nhất), Minimax duyệt từ 74 nodes (depth 1) lên tới **2.032.724 nodes** (depth 4) – tăng gần 27.000 lần.
- Alpha-Beta Pruning:** Giảm đáng kể số trạng thái nhờ cắt tỉa. Tỷ lệ cắt tỉa tăng mạnh theo độ sâu:
 - Ở depth 1: Không có cắt tỉa (tỷ lệ 1.00x) vì chỉ duyệt 1 lớp.
 - Ở depth 2: Cắt tỉa trung bình 1.7–4.0 lần.
 - Ở depth 3: Cắt tỉa trung bình 3.5–12.1 lần.
 - Ở depth 4: Cắt tỉa **3.8–51.0 lần**. Đặc biệt State 6 đạt hiệu quả cắt tỉa cao nhất 51.03x (Minimax: 2.032.724 nodes vs Alpha-Beta: 39.831 nodes).
- Ảnh hưởng của trạng thái bàn cờ:**
 - State 1 (bàn cờ trống):** Chỉ có 1 nước đi hợp lệ (ô trung tâm) ở nước đầu tiên, nên số nodes rất ít (2–1.298 nodes).
 - State 2, 5 (giữa trận, rải rác):** Branching factor cao hơn (15–25 nước đi), số nodes tăng mạnh. State 5 ở depth 4 Minimax duyệt tới 302.295 nodes.
 - State 3 (O có 3 quân):** Score = 100.000 ở mọi độ sâu, AI phát hiện ngay cơ hội thắng. Alpha-Beta cắt tỉa rất hiệu quả (34.94x ở depth 4) vì tìm thấy nước đi thắng sớm.

- **State 6 (phức tạp, 9 quân):** Branching factor lớn nhất, Minimax ở depth 4 duyệt hơn 2 triệu nodes, mất **130 giây**. Alpha-Beta giảm xuống chỉ còn 39.831 nodes (**2.5 giây**).

11.2 Thời gian chạy

1. **Tương quan với số nodes:** Thời gian chạy tỷ lệ thuận với số trạng thái đã duyệt. Ví dụ State 2 ở depth 4: Minimax mất 7.75s (140.324 nodes) vs Alpha-Beta mất 0.50s (9.023 nodes) – tỷ lệ thời gian (15.45x) gần bằng tỷ lệ nodes (15.55x).
2. **Chi phí sao chép:** Mỗi nước đi giả lập sử dụng `game.copy()` (deepcopy toàn bộ ma trận 9×9), chiếm khoảng 30–40% thời gian thực thi.
3. **Alpha-Beta nhanh hơn rõ rệt ở depth cao:**
 - State 5, depth 4: Minimax 16.07s vs Alpha-Beta 0.67s (nhanh hơn **24x**).
 - State 6, depth 4: Minimax 130.43s vs Alpha-Beta 2.52s (nhanh hơn **52x**).
4. **Giới hạn thực tế:** Với Alpha-Beta, depth 3 cho thời gian phản hồi $< 0.3s$ trên mọi trạng thái. Depth 4 vẫn chấp nhận được ($< 2.5s$). Minimax thuần ở depth 4 đã quá chậm (tối 130s ở State 6).

12 Nhận xét về ảnh hưởng của độ sâu tìm kiếm

12.1 Lựa chọn thuật toán theo mức độ khó

Chương trình không sử dụng cùng một thuật toán cho tất cả các mức độ khó. Thay vào đó, hệ thống **chủ động lựa chọn thuật toán** phù hợp với từng mức độ dựa trên sự cân bằng giữa tốc độ phản hồi và chất lượng nước đi:

Bảng 10: Cấu hình thuật toán theo mức độ khó

Mức độ	Thuật toán	Độ sâu	Lý do lựa chọn
Dễ	Minimax thuần	1	Không gian nhỏ, không cần cắt tỉa
Trung bình	Minimax thuần	2	Nodes ít (~ 300), Minimax đủ nhanh
Khó	Alpha-Beta Pruning	3	Cần cắt tỉa để giảm từ ~ 5.000 xuống ~ 1.400 nodes
Cực khó	Alpha-Beta Pruning	4	Bắt buộc cắt tỉa: giảm từ $\sim 100K$ xuống $\sim 11K$ nodes

Cài đặt trong mã nguồn (`play_caro.py`, hàm `start_game`):

```

1 def start_game(self):
2     level = self.level_var.get()
3     if level == "easy":
4         self.ai_depth = 1
5         self.ai_algorithm = 'minimax'           # Minimax thuần
6     elif level == "medium":
7         self.ai_depth = 2

```

```

8         self.ai_algorithm = 'minimax'           # Minimax thuần
9     elif level == "hard":
10         self.ai_depth = 3
11         self.ai_algorithm = 'alphabeta'         # Alpha-Beta Pruning
12     else: # expert
13         self.ai_depth = 4
14         self.ai_algorithm = 'alphabeta'         # Alpha-Beta Pruning

```

Listing 10: Lựa chọn thuật toán theo mức độ khó

Nguyên tắc thiết kế:

- **Mức Dễ & Trung bình** sử dụng **Minimax thuần** vì ở độ sâu 1–2, không gian tìm kiếm nhỏ (2–324 nodes), thời gian chạy < 0.02s. Việc thêm Alpha-Beta không cải thiện đáng kể và chi phí quản lý α, β là không cần thiết.
- **Mức Khó & Cực khó bắt buộc** sử dụng **Alpha-Beta Pruning** vì ở độ sâu 3–4, Minimax thuần phải duyệt hàng nghìn đến hàng triệu trạng thái (State 6: 2.032.724 nodes, mất 130s). Alpha-Beta cắt tỉa giúp giảm 5–51 lần, đưa thời gian phản hồi về mức chấp nhận được (< 3s).

12.2 Chất lượng nước đi

- **Độ sâu 1 (Dễ – Minimax):** AI chỉ nhìn trước 1 nước, đánh giá theo heuristic tức thì. AI thiếu khả năng phòng thủ và dễ bị đánh bại.
- **Độ sâu 2 (Trung bình – Minimax):** AI nhìn trước 2 nước (1 nước mình + 1 nước đối thủ). Bắt đầu có khả năng phòng thủ cơ bản nhưng chưa tạo được thế tấn công phức tạp.
- **Độ sâu 3 (Khó – Alpha-Beta):** AI nhìn trước 3 nước, có thể tạo các bẫy 2 nhánh (double threat). Đây là mức cân bằng tốt giữa hiệu suất và chất lượng.
- **Độ sâu 4 (Cực khó – Alpha-Beta):** AI nhìn trước 4 nước, gần như không thể bị đánh bại bởi người chơi bình thường. Tuy nhiên, thời gian tính toán tăng đáng kể.

12.3 Mối quan hệ giữa độ sâu và tài nguyên

Khi tăng độ sâu lên 1, số trạng thái duyệt tăng lên khoảng b lần (với b là branching factor):

Bảng 11: Ảnh hưởng của độ sâu đến tài nguyên (Alpha-Beta, trung bình qua 6 trạng thái)

Độ sâu	Mức độ	Nodes TB	Thời gian TB	Nhận xét
1	Dễ	~33	< 0.002s	Phản hồi tức thì, AI yếu
2	Trung bình	~156	< 0.01s	Phản hồi nhanh, AI trung bình
3	Khó	~1.424	0.08s	Chấp nhận được, AI khá
4	Cực khó	~11.368	0.68s	Hơi chậm, AI rất mạnh

12.4 Kết luận

Độ sâu 3 kết hợp Alpha-Beta Pruning là lựa chọn cân bằng tối ưu cho gameplay, vừa đảm bảo AI đủ mạnh, vừa cho thời gian phản hồi nhanh.

13 Ưu điểm và hạn chế của chương trình

13.1 Ưu điểm

1. **Cài đặt đầy đủ hai thuật toán:** Minimax và Alpha-Beta Pruning, cho phép so sánh hiệu suất trực tiếp.
2. **Sinh nước đi thông minh:** Chỉ xét các ô lân cận, giảm đáng kể không gian tìm kiếm so với duyệt toàn bộ bàn cờ.
3. **Hàm đánh giá đa cấp:** Nhận biết nhiều mẫu hình (4 liên tiếp, mở 3, chặn 3, mở 2...) và có bonus vị trí trung tâm.
4. **Giao diện đồ họa đẹp mắt:** Phong cách Neon hiện đại sử dụng Tkinter, có hiệu ứng hover, hiển thị thông tin AI (score, nodes, time).
5. **Đa luồng (Multi-threading):** AI tính toán trên luồng riêng, giao diện không bị đóng băng khi AI suy nghĩ.
6. **Nhiều trạng thái thử nghiệm:** 6 trạng thái test case đa dạng để đánh giá hiệu suất toàn diện.
7. **Benchmark tích hợp:** Hàm `run_benchmark` và `compare_algorithms` cho phép so sánh tự động hai thuật toán.
8. **4 mức độ khó:** Cho phép người chơi ở mọi trình độ đều có trải nghiệm phù hợp.

13.2 Hạn chế

1. **Chi phí sao chép bàn cờ:** Sử dụng `deepcopy` cho mỗi nước đi giả lập gây tốn bộ nhớ và thời gian. Có thể cải thiện bằng kỹ thuật `make/unmake` (đặt quân rồi gỡ quân).
2. **Chưa có bảng chuyển vị (Transposition Table):** Phiên bản `ai_agent.py` chưa sử dụng bảng chuyển vị để tránh đánh giá lại các trạng thái đã gặp (file `AI.py` có cài đặt nhưng chưa tích hợp).
3. **Chưa có Iterative Deepening:** Không tìm kiếm từ độ sâu thấp lên cao, mất cơ hội sắp xếp nước đi tốt hơn cho cắt tỉa Alpha-Beta.
4. **Hàm đánh giá chưa hoàn hảo:** Có thể bỏ sót một số mẫu hình phức tạp (ví dụ: double open three, fork threats).
5. **Giới hạn độ sâu:** Độ sâu 5 trở lên gây thời gian chờ quá lâu, không phù hợp cho gameplay thời gian thực.

6. **Chưa có chế độ PvP:** Không hỗ trợ chế độ hai người chơi với nhau (chỉ có Human vs AI).
7. **Kích thước bàn cờ cố định:** Bàn cờ cố định 9×9 , chưa hỗ trợ tùy chỉnh kích thước.

13.3 Hướng phát triển

- Tích hợp Transposition Table với Zobrist Hashing (đã có sẵn trong `AI.py`).
- Thêm Iterative Deepening để cải thiện move ordering.
- Tối ưu hàm đánh giá với nhiều pattern phức tạp hơn.
- Thêm chế độ PvP và tùy chỉnh kích thước bàn cờ.
- Áp dụng Machine Learning để cải thiện hàm đánh giá.

14 Tài liệu tham khảo và phần đã tham khảo từ các repo mẫu

14.1 Tài liệu tham khảo

1. S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson, 2020.
Chương 5: Adversarial Search and Games – cơ sở lý thuyết cho Minimax và Alpha-Beta Pruning.
2. A. Neto, *Minimax Algorithm with Alpha-Beta Pruning*, GeeksforGeeks, 2023.
URL: <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-4-alpha-beta-pruning/>
3. Wikipedia contributors, *Gomoku*, Wikipedia, The Free Encyclopedia.
URL: <https://en.wikipedia.org/wiki/Gomoku>
4. Python Software Foundation, *tkinter — Python interface to Tcl/Tk*, Python Documentation.
URL: <https://docs.python.org/3/library/tkinter.html>

14.2 Phần đã tham khảo từ repo mẫu

1. **Cấu trúc Pattern Dictionary** (`utils.py`): Hệ thống bảng điểm pattern được tham khảo từ các dự án Gomoku AI mã nguồn mở, trong đó các mẫu hình cờ (4 quân liên tiếp, mở 3, chặn 3, mở 2...) được gán điểm số theo thang đánh giá phân cấp.
2. **Zobrist Hashing** (`utils.py`, `AI.py`): Kỹ thuật băm trạng thái bàn cờ bằng phép XOR được tham khảo từ lý thuyết Zobrist Hashing phổ biến trong các engine cờ vua/cờ caro.
3. **Cấu trúc thuật toán Minimax và Alpha-Beta** (`ai_agent.py`): Cài đặt dựa trên pseudocode từ sách giáo khoa AIMA (Artificial Intelligence: A Modern Approach) của Russell & Norvig, được điều chỉnh cho phù hợp với bài toán cờ Caro.

4. **Giao diện đồ họa Neon** (`play_caro.py`): Thiết kế giao diện sử dụng Tkinter Canvas được phát triển riêng, lấy cảm hứng từ phong cách thiết kế Neon/Cyberpunk trong các dự án UI hiện đại.

Kết luận

Bài tập lớn đã hoàn thành việc xây dựng chương trình chơi Cờ Caro với AI sử dụng thuật toán Minimax và Alpha-Beta Pruning. Qua quá trình thực hiện, các kết quả chính đạt được:

1. Hiểu và cài đặt thành công hai thuật toán tìm kiếm đối kháng cổ điển.
2. Chứng minh bằng thực nghiệm rằng Alpha-Beta Pruning giảm đáng kể số trạng thái cần duyệt (trung bình 10–30 lần) so với Minimax thuần.
3. Thiết kế hàm đánh giá heuristic với nhiều mẫu hình, giúp AI có khả năng chơi ở mức khá đến rất mạnh tùy theo độ sâu.
4. Xây dựng giao diện đồ họa thân thiện với người dùng.

Chương trình có thể được mở rộng thêm với các kỹ thuật tối ưu nâng cao như Transposition Table, Iterative Deepening, và Move Ordering để cải thiện hiệu suất tìm kiếm ở các độ sâu lớn hơn.